



Sir Syed University of Engineering & Technology, Karachi

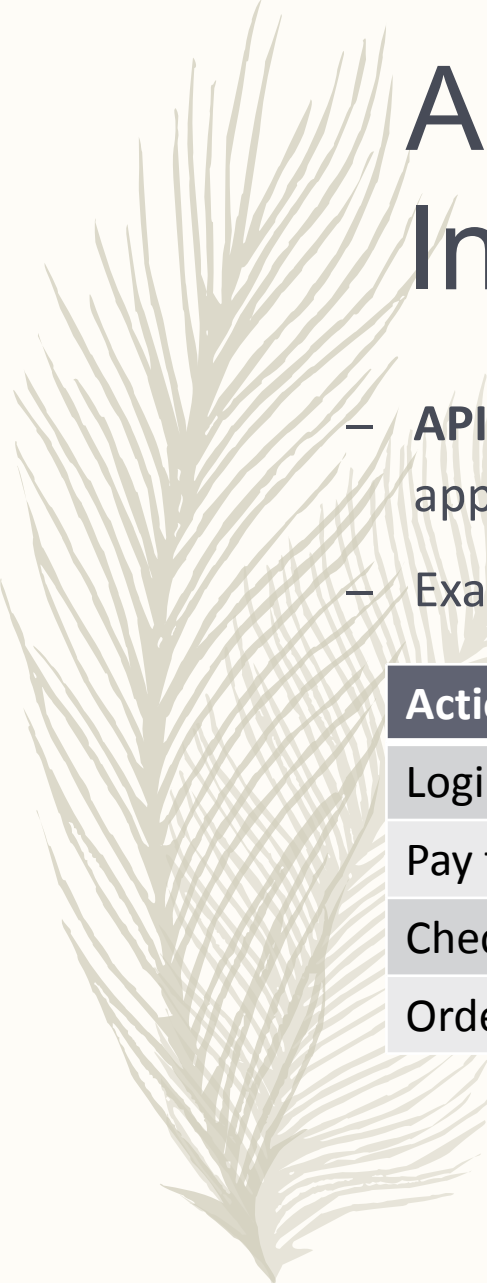
Mobile Application Development

Lecture 7: Working with API's



Objective

- API's
- Fetching data API's
 - fetch()
 - Axios
- API Response
- Async/Await
- FlatList



API (Application Programming Interface)

- **API** is a set of rules that allows one application to request data or services from another application.
- Examples:

Action You Perform	API Working Behind the Scenes
Login with Google	Google Auth API
Pay through JazzCash/Easypaisa	Payment API
Check weather in an app	Weather API fetches live data
Order food from Foodpanda	Restaurant data through APIs



Fetching data API's

- React Native uses below two methods for fetching data
 - **built-in fetch()** method
 - **Axios** library
- Both are async operations
- Axios is a popular library that simplifies HTTP requests and automatically parses JSON.

fetch() Example

```
import React, { useEffect, useState } from 'react';
import { View, Text, FlatList, ActivityIndicator }
from 'react-native';

type User = {
  id: number;
  name: string;
  email: string;
};

export default function App() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);
```

```
const fetchUsers = async () => {
  try {
    const response = await
    fetch('https://jsonplaceholder.typicode.com/users');

    const data: User[] = await response.json();
    setUsers(data);
  } catch (error) {
    console.error('Error fetching users:', error);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchUsers(); // function called here
}, []);
```

fetch() Example

```
if (loading) {  
  return <ActivityIndicator size="large" color="#6200EE" />;  
}  
  
return (  
  <View style={{ flex: 1, padding: 20 }}>  
    <FlatList  
      data={users}  
      keyExtractor={({item}) => item.id.toString()}  
      renderItem={({ item }) => (  
        <Text style={{ padding: 10, fontSize: 16 }}>  
          {item.name} {item.email}  
        </Text>  
      )}  
    </FlatList>  
  </View>  
);  
}
```



fetch() Example

- `fetch(url)` returns a Promise.
- `.json()` parses the response as JSON.
- Use `async/await` for clean syntax.
- Wrap fetching logic in `useEffect` to run on component mount.
- Handle loading state with `ActivityIndicator`

Axios Example

```
import React, { useEffect, useState } from 'react';
import { View, Text, FlatList, ActivityIndicator } from
'react-native';
import axios from 'axios';

type User = {
  id: number;
  name: string;
  email: string;
};

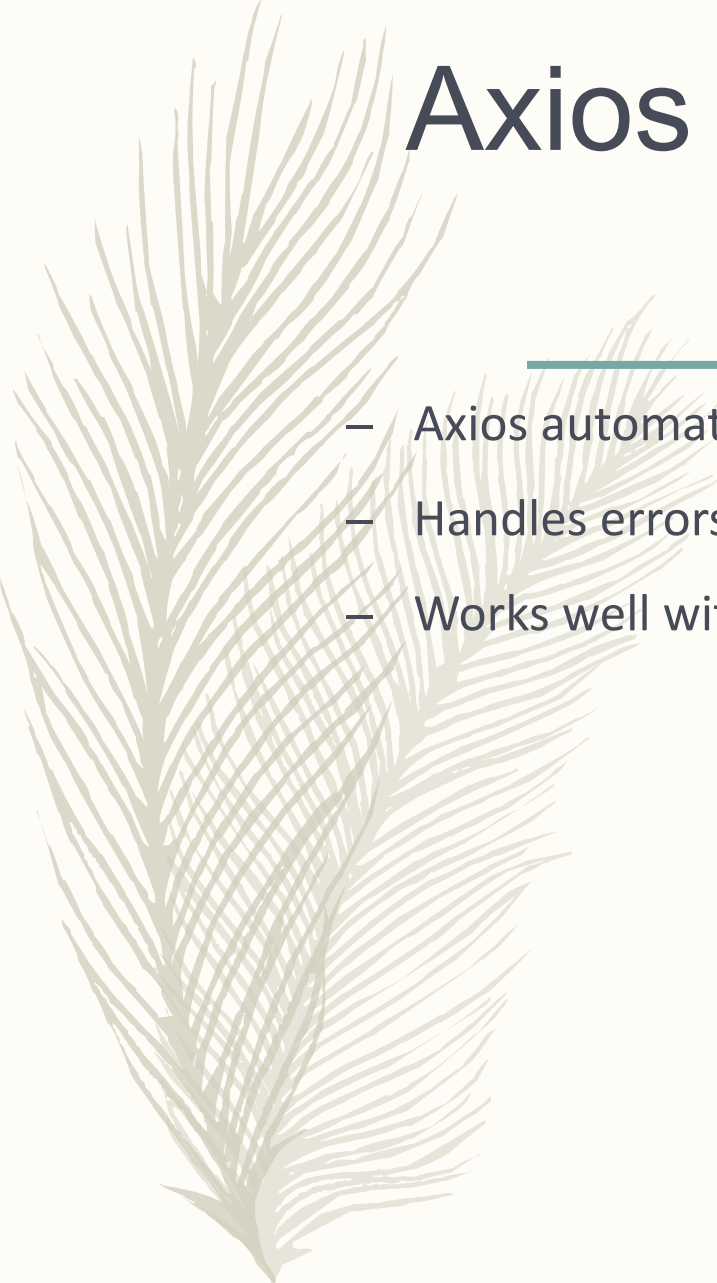
export default function App() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
    const fetchUsers = async () => {
      try {
        const response = await
        axios.get<User[]>('https://jsonplaceholder.typicode.com/
        users');

        setUsers(response.data);
      } catch (error) {
        console.error('Error fetching users:', error);
      } finally {
        setLoading(false);
      }
    };
    fetchUsers();
  }, []);
```


Axios Example

```
if (loading) {  
  return <ActivityIndicator size="large" color="#6200EE" />;  
}  
  
return (  
  <View style={{ flex: 1, padding: 20 }}>  
    <FlatList  
      data={users}  
      keyExtractor={({item}) => item.id.toString()}  
      renderItem={({ item }) => (  
        <Text style={{ padding: 10, fontSize: 16 }}>  
          {item.name} {item.email}  
        </Text>  
      )}  
    </FlatList>  
  </View>  
);
```



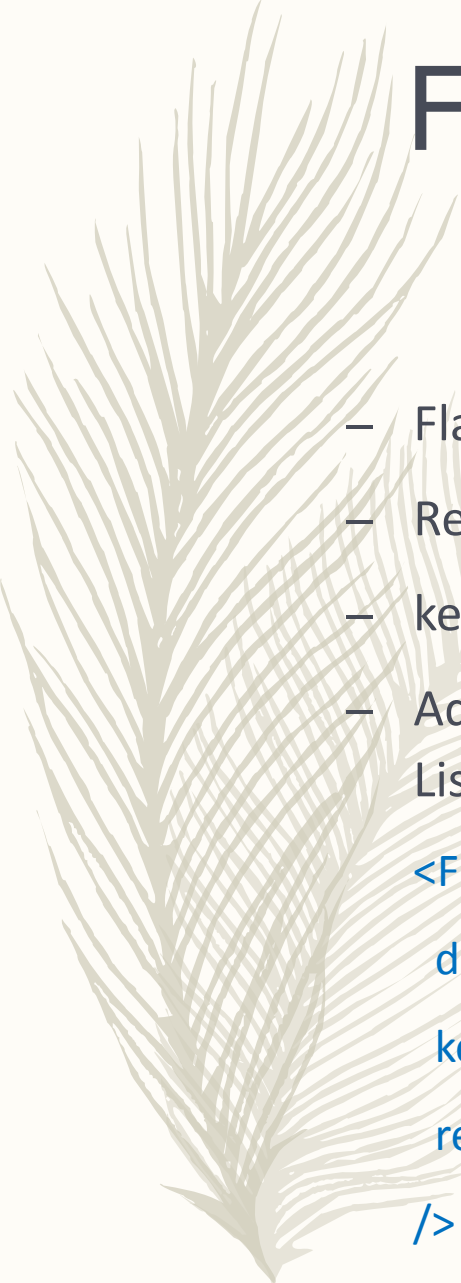
Axios Example

- Axios automatically converts response to JSON, no need for .json().
- Handles errors more gracefully.
- Works well with TypeScript types



JSON Responses

- Most APIs return JSON.
- Using `await response.json()` (with `fetch`) or `response.data` (with `Axios`) converts it to a JavaScript object/array.
- Always handle errors using `try/catch`.



Flat List

- FlatList is optimized for large lists in React Native.
- Requires data and renderItem props.
- keyExtractor helps React efficiently re-render items
- Additional props which can be used are ItemSeparatorComponent, ListHeaderComponent / ListFooterComponent

<FlatList

data={users} // array of objects

keyExtractor={({item}) => item.id.toString()} // unique key

renderItem={({ item }) => <Text>{item.name}</Text>}

/>